

Question 1

- a) The *Turing Test*, introduced by Alan Turing in 1950, is a widely recognised test in the field of Artificial Intelligence (AI). In contrast, the *Total Turing Test*, which was introduced by Stevan Harnad in 1991, serves as a more rigorous benchmark for AI.

- (i) Describe the purpose of the *Turing Test* in your own words. (1 mark)

The Turing Test is a test used for evaluating a computer's ability to demonstrate human-like intelligence.

- (ii) Describe how the *Turing Test* works and the significance of its outcome in your own words. (3 + 1 marks)

- The test involves a computer, a human respondent, and a human interrogator, ensuring no direct physical interaction
- The interrogator communicates with both the computer and the human respondent solely through a terminal.
- The computer passes the test if, after asking a series of written questions, the interrogator cannot distinguish whether the responses are from a human or a computer.
- Any computer that passes the test is considered to exhibit human-like intelligence.

- (iii) Identify **TWO (2)** additional requirements of the *Total Turing Test* compared to the original *Turing Test*. (2 marks)

- Perceptual ability
 - The AI must be able to perceive its environment using sensory inputs such as vision, sound, or touch.
 - *Example: A robot with cameras and microphones can recognize a person's face and tone of voice before answering questions.*

- Object manipulation ability
 - The AI must be able to physically interact with and manipulate objects in its environment.
 - *Example: A household robot being able to pick up a cup, pour water, and hand it to a person.*

(iv) Describe the **SIX (6)** capabilities a computer must possess to pass the *Total Turing Test*.
(6 marks)

- To communicate in human languages (or natural language processing)
- To store what it knows or hears (or knowledge representation)
- To do reasoning and to draw new conclusions based on rules and data (or automated reasoning)
- To adapt to new circumstances and handle uncertainties through the process of learning data (or machine learning)
- To derive visual information from its percepts (or computer vision)
- To integrate AI into machines that can sense, plan, and act in the physical world. (or robotics)

- b) In AI, an environment can be thought of as the world in which an intelligent entity, usually known as agent, operates and makes decisions. Since the range of environments in AI is vast, they are categorised into the following six dimensions to guide the design of agents as well as determining the applicability agent implementation:
- Single agent vs. multiagent
 - Fully observable vs. partially observable
 - Deterministic vs. non-deterministic
 - Episodic vs. sequential
 - Static vs. dynamic
 - Discrete vs. continuous

Google Photos, a *facial recognition application*, can automatically group photos based on the people in them, and help users organise and search their photo library. Suppose its environment is images. Identify the properties of the environment based on the **SIX (6)** dimensions mentioned earlier and justify your answer.
(6 + 6 marks)

Property	Justification
Fully observable	The agent can make an optimal decision because it can "see" the entire image the whole time.
Single agent	The agent is solving face recognition problem by itself.
Deterministic	The outcome of tagging people or faces in an image is always the same when the same task is repeated.
Episodic	The result of face recognition for an image has no impact on the result of face recognition for the next image.
Static	When the agent is screening the image for percept or analysing the image trying to identify faces, the image remains unchanged.
Discrete	Regardless of the resolution of the image be processed, there is a finite set of percepts in terms of pixels.

Question 2

- a) In AI, *classical search* and *local search* are two common strategies for solving search problems, with the latter often used for optimisation.
- (i) Provide **TWO (2)** examples for each type of search strategy. (4 marks)

Classical Search

- Breadth-first search
- Uniform-cost search
- Depth-first search
- Backtracking search
- Depth-limited search
- Iterative deepening search
- Bidirectional search

Local Search

- Stochastic hill climbing
- First-choice hill climbing
- Random-restart hill climbing
- Simulated annealing
- Local beam search
- Evolutionary Algorithms

- (ii) Differentiate between these two types of strategies in **TWO (2)** distinct ways. (4 marks)

	Classical Search	Local Search

Solution	Return a sequence of actions as a solution.	Return a single final state as a solution.
Optimality	CSS finds an optimal solution.	LSS finds a good enough solution without guaranteeing optimality.
Space Consumption	CSS consume more space as they track both expanded and unexpanded nodes.	LSS uses less space by focusing only on the current state.
Search Pattern	CSS explores the search space systematically (like checking every possible road in order)	LSS moves heuristically between states without exhaustive exploration.

b) *Hill climbing* is a popular local search technique, especially useful for optimisation problems.

(i) Explain what *hill climbing* is in your own words. (3 marks)

Hill climbing is a local search algorithm that **keeps only the current state** , **moves to the best neighbouring state** , and **stops when no better neighbour exists** .

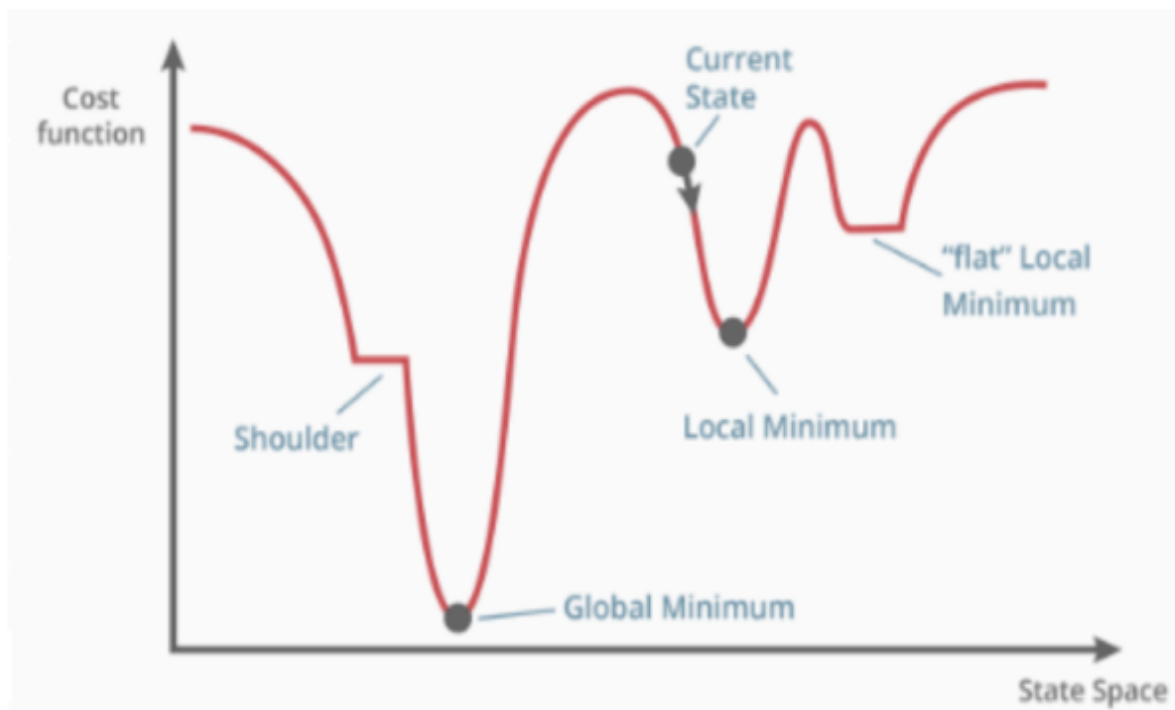
(ii) Explain **THREE (3)** advantages of using *hill climbing*. (3 marks)

- Can often make rapid progress toward a solution.
 - Quickly improves the current state by moving to better neighbors.
- An efficient solution to improve a bad state.
 - Even if you start with a poor solution, it can make it much better step by step.

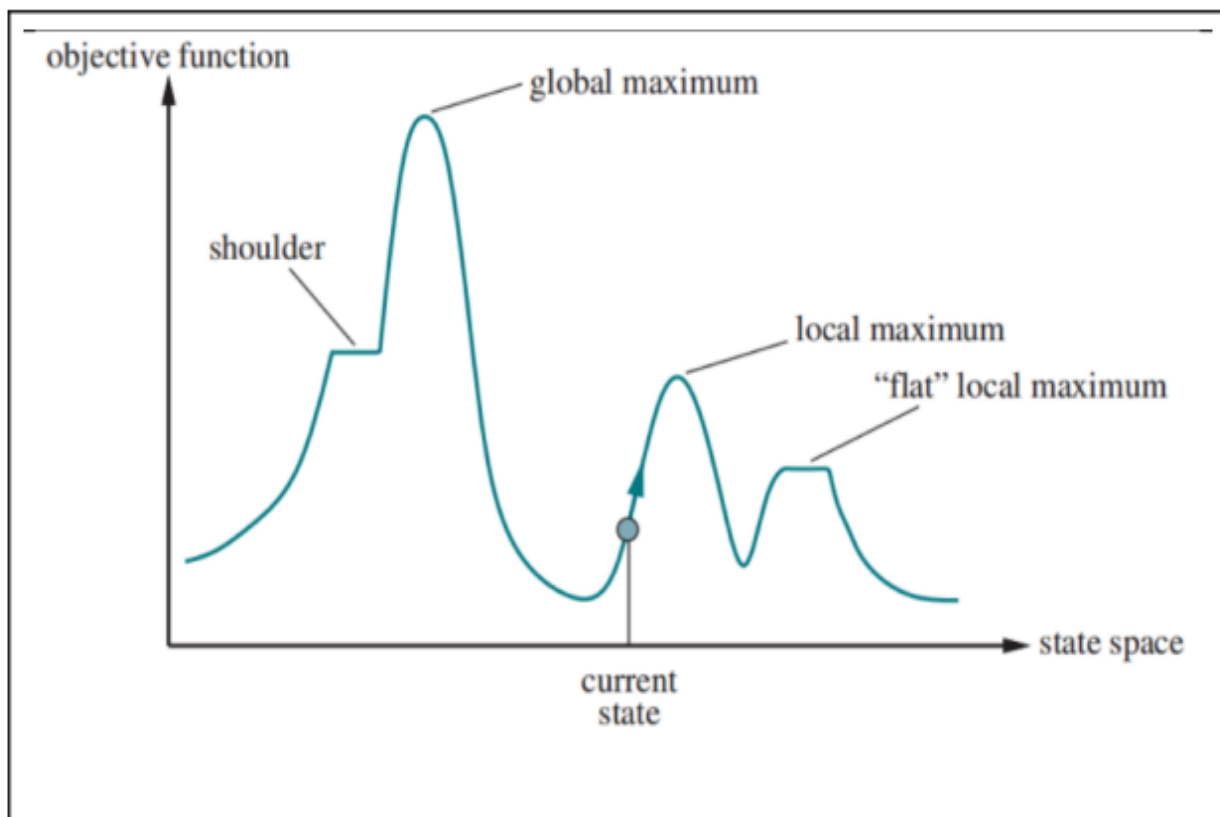
- Requires low memory consumption.
 - Only needs to store the current state (no big search tree).
- Requires low computational power.
 - Simple comparisons make it lightweight and fast to run.

- (iii) Propose **ONE (1)** *heuristic cost function* and **ONE (1)** *heuristic objective function* for applying *hill climbing* to solve the *8-Queens problem*. Then, illustrate a *state-space landscape* for each function, where the elevation corresponds to the respective function. (Remark: You must label the current state, shoulder, local minimum/maximum, global minimum/maximum, as well as the x-axis and y-axis correctly.) (2 + 7 marks)

Heuristic cost function: The number of pairs of queens that are attacking each other, either directly or indirectly.



Heuristic objective function: The number of non-attacking pairs of queens, either directly or indirectly.



- (iv) Based on the two 8-Queens problem states provided in **Figure 1** and **Figure 2**, compute their heuristic values using the *heuristic cost function* proposed in the **Question 2 b) (iii)**. (2 marks)

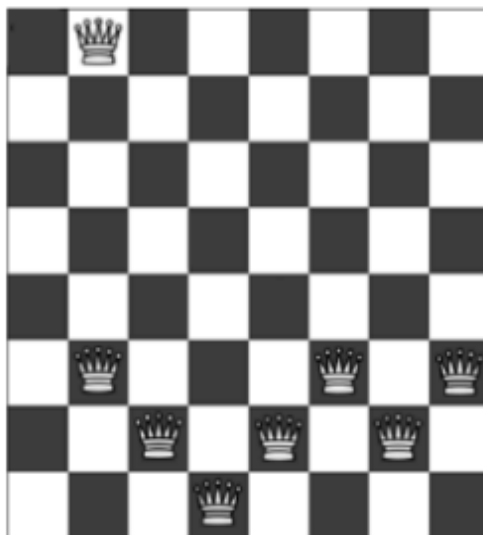


Figure 1. State x of 8-Queen problem

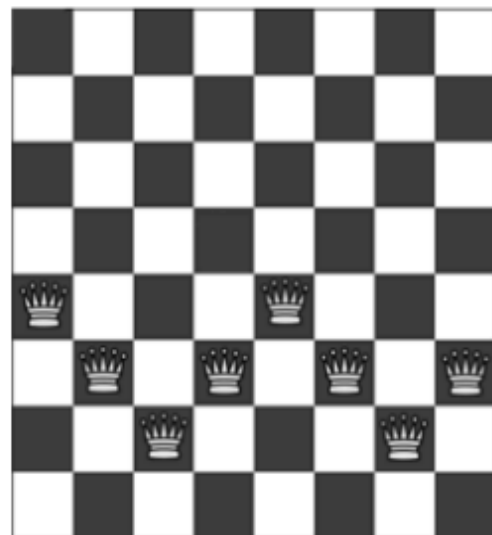


Figure 2. State y of 8-Queen problem

Heuristic value (Figure 1), $h = 15$

Heuristic value (Figure 2), $h = 18$

Question 3

Adversarial search aims to find optimal strategies in environments where multiple agents compete. It is widely used in competitive games like chess and tic-tac-toe. Two common adversarial strategies are *minimax search* and *alpha-beta pruning*.

- a) Explain what *minimax search* is in your own words. (2 marks)

The minimax is a decision-making algorithm used in games that computes the optimal move based on the assumption that the opponent plays their best (or optimally).

- b) Explain what *alpha-beta pruning* is in your own words. (2 marks)

The alpha-beta pruning is an optimization technique that improves the minimax algorithm by pruning removes unnecessary branches without altering the final decision returned by the minimax.

- c) Answer the following questions based on a simple *game tree* provided in **Figure 3**.

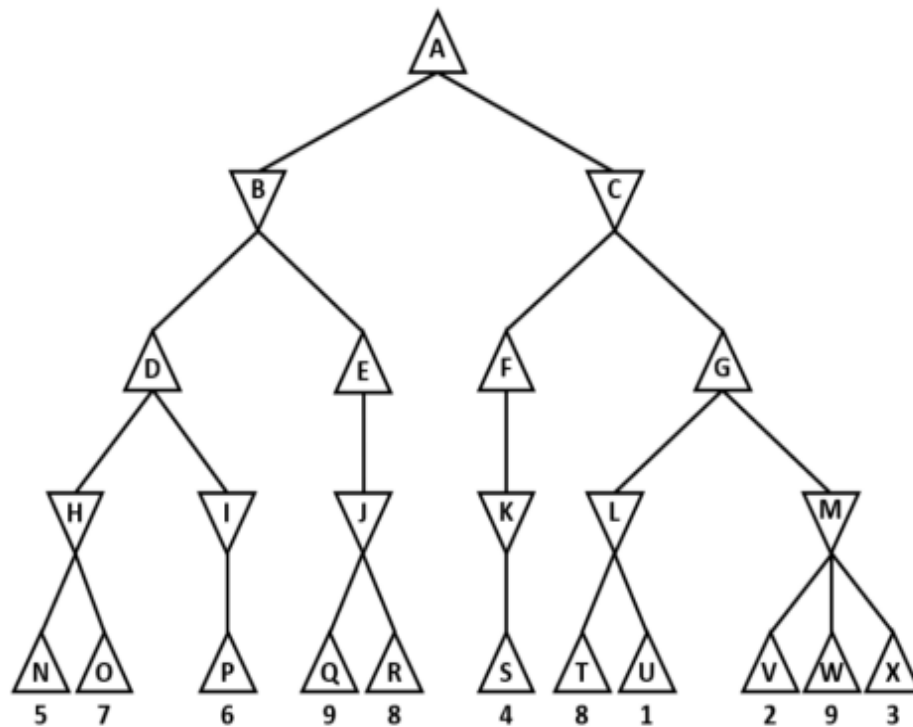


Figure 3. A game tree

- (i) Compute the *minimax* value for 12 nodes from A to M. (6 marks)

6	2	6	8	4	2	5	6	8	4	1	2

- (ii) Identify the best move for the *MAX* player at the root node. (1 mark)

A move that leads to B because B has the highest minimax value.

- (iii) Apply the *minimax search* with *alpha-beta pruning* in *left-to-right* order and identify the nodes that would be pruned. Then, apply the *minimax search* with *alpha-beta pruning* in *right-to-left* order and identify the nodes that would be pruned. (**Remark:** For each order, you must demonstrate how alpha-beta pruning works on the game tree provided in **Figure 3**.) (13 marks)

- Left-to-right order: G, L, M, T, U, V, W, X

- Right-to-left order: T

(iv) Derive a conclusion based on your answers to **Question 3 c) (iii)**. (1 mark)

The effectiveness of alpha-beta pruning is highly dependent on the order in which the states are examined.

Question 4

Sudoku is a puzzle game that involves the logical placement of numbers and has introduced millions to the concept of *Constraint Satisfaction Problems (CSP)*. **Figure 4** shows an example of a 4×4 Sudoku grid consisting of 16 squares, some of which are pre-filled with digits between 1 and 4. The objective is to fill the grid so that each row, column, and 2×2 sub-grid contains the digits 1 through 4. Suppose the *Alldiff* global constraint is applied to solve the Sudoku in **Figure 4**. Answer the following questions:

	1	2	3	4
A	1			4
B			3	
C	3			2
D		4	1	3

Figure 4. State x of the 4×4 Sudoku.

- a) Explain what *CSP* is in your own words. (3 marks)

- A CSP is a problem of **finding values for variables that satisfy all given constraints** , where **each variable must take a value from its defined domain** . When all constraints between **variables are satisfied** , a solution is found.
- It consists of three main components: 1) **variables**, 2) **domains**, and 3) **constraints**.

- b) Explain the reasoning behind the name *Alldiff*. (1 mark)

All variables involved in the constraint must have different values.

- c) Provide **TWO (2)** other examples of global constraints. (2 marks)

Resources constraint

- Means that some limited resource (time, money, people, machines, energy, etc.) must not be exceeded.
- Common in scheduling, planning, and project management.
- *Example:*
 - *In a factory, only 3 machines are available, so at most 3 jobs can run at the same time.*
 - *In project planning, you have 10 workers total, so you can't assign 15 tasks at once.*
 - *Representation: The total hours of all tasks ≤ 40*

Bound constraint

- Means a variable's value must stay within a minimum and maximum limit.
- Like a fence around the possible values.
- *Example:*
 - *Temperature in a chemical process must stay between 20°C and 80°C.*
 - *Exam duration must be at least 1 hour but no more than 3 hours.*
 - *Representation: $\min \text{ budget} < \text{The total cost must be} \leq \max \text{ budget}$ (Since the bound applies to a sum/aggregate of several variables, it's a global constraint.)*

- d) Design the Sudoku as a CSP by identifying its *variables*, the *domain* for each variable, and the *constraints* using the *Alldiff* constraint. (9 marks)

Variable, X =

{A1, A2, A3, A4,

B1, B2, B3, B4,

C1, C2, C3, C4

D1, D2, D3, D4}

Domain of empty squares, $D_i = \{1, 2, 3, 4\}$.

Constraints

- Alldiff (A1, A2, A3, A4)
- Alldiff (B1, B2, B3, B4)
- Alldiff (C1, C2, C3, C4)
- Alldiff (D1, D2, D3, D4)
- Alldiff (A1, B1, C1, D1)
- Alldiff (A2, B2, C2, D2)
- Alldiff (A3, B3, C3, D3)
- Alldiff (A4, B4, C4, D4)
- Alldiff (A1, A2, B1, B2)
- Alldiff (A3, A4, B3, B4)
- Alldiff (C1, C2, D1, D2)
- Alldiff (C3, C4, D3, D4)

- e) To simplify implementation, suppose all *Alldiff* constraints are expanded into binary constraints, allowing the *AC-3 algorithm* (the most popular algorithm for enforcing *arc consistency*) to be applied directly. Explain how the value of B1 is determined by making following variables *arc-consistent* step by step.

(i) B4

(2 marks)

The column constraint enforces the removal of 2, 3 and 4 from B4's domain, leaving B1 with a domain of {1}

(ii) A3

(2 marks)

The box constraint enforces the removal of 1, 3 and 4 from A3's domain, leaving A3 with a domain of {2}

(iii) A2

(2 marks)

The row constraint enforces the removal of 1, 2 and 4 from A2's domain , leaving A2 with a domain of {3}

(iv) D1

(2 marks)

The row constraint enforces the removal of 1, 3 and 4 from D1's domain , leaving D1 with a domain of {2}

(v) B1

(2 marks)

The column constraint enforces the removal of 1, 2 and 3 from B1's domain , leaving B1 with a domain of {4}